

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Weeks 1-3 Lab Assignment

Image Pixels, Filters, Color Channels, and Edges in Python

Student Name:	Muhammad Yousif
Father's Name:	Mahmood Babar
Roll Number:	2k24/cse/108
Course Work:	Individual Lab Project
Deadline:	30 September 2026

Submitted to Private Course Repository
Tools: Python, OpenCV (cv2), NumPy, Matplotlib

Task 1: Image Inspection & Properties

Objective: Load the captured photograph programmatically, extract image dimensions, channel count, NumPy shape, total pixel count, and memory allocation size.

The original image is read directly from `images/original.jpg` without GUI popups. Below is the implemented function signature and data breakdown:

```
def inspect_image(image_path: str) -> dict: """Load the image and return its measured image-data properties.""" # Reads image using cv2.imread and extracts shape, dtype, and color channels.
```

Property Measured	Observed Value / Format	Explanatory Notes
Dimensions (Width × Height)	1920 × 1080 pixels	Full resolution landscape framing
Channels & Color Order	3 Channels (BGR Format)	OpenCV default channel ordering
NumPy Array Shape	(1080, 1920, 3)	(Height, Width, Channel Count)
Total Pixel Count	2,073,600 pixels	Width × Height
Estimated Size (8-bit)	~6,220,800 bytes (~6.22 MB)	Computed as pixels × channels

Task 2: Color Channels & Resolution Scaling

Objective: Isolate individual color planes (Red, Green, Blue), convert to grayscale, and downsample dimensions to half-size. Saved to `outputs/pixel_views.png`.

Splitting channels explicitly reveals intensity distributions across spectral bands. The downsampled view demonstrates spatial resolution reduction by factor of 2.

- **Red Channel View:** Displays high reflection where red pigments are present on everyday objects.
- **Green & Blue Channels:** Show distinct tonal gradients corresponding to ambient lighting and surface absorption.
- **Downsampled View:** Dimensions reduced to 960 × 540 pixels. High-frequency fine details (such as edge textures on the handwritten verification card) are noticeably smoothed out.

Task 3 & 4: Image Adjustments, Blur & Edges

Task 3 Adjustments: Brighter (+40 delta), higher contrast ($\times 1.5$ factor), and binary thresholding (threshold = 127) saved to `outputs/adjustments.png`.

Pixel values are securely clipped to `[0, 255]`. Thresholding cleanly separates high-intensity background elements from dark text on the handwritten card ("HCCGI Image Lab" with initials).

Task 4 Blur & Edges: Mean blur kernel size = 5 applied. Sobel edge maps computed for original and blurred grayscale images, saved to `outputs/blur_and_edges.png`.

Observation: Applying the mean blur filter acts as a low-pass filter, effectively reducing noise and suppressing minor textural gradients. Consequently, the subsequent Sobel edge detection on the blurred image yields cleaner, thicker, but less sharp edge lines compared to edge detection on the raw noisy grayscale image.

Task 5: Reproducibility & Submission Script

The script runs completely from the repository root via terminal command:

```
python3 weeks01-03_image_lab.py
```

All function names, parameters, and output paths comply strictly with requirements. Dependencies are fully specified in `requirements.txt` containing `opencv-python`, `numpy`, and `matplotlib`.